

Loan Default

Medallion Architecture Strategy & Implementation

TL;DR

A monthly Bronze→Silver→Gold pipeline that turns 4 raw CSVs into an ML-ready feature store and label store for predicting loan default at application time.

By Jeslyn Wangsa · May 2026

The problem we're solving

What we need, what 'done' looks like, who reads this deck

Business problem

- Bank lends cash loans; some borrowers default.
- We need to score default risk at the moment of loan application, before any repayment data exists.
- An ML model will do the scoring (next assignment); this project ships the data pipeline that feeds it.
- Audience: ML engineers (will train the model), data engineers (will operate the pipeline), risk & business stakeholders (need to trust the inputs).

Definition of done

- Three medallion layers — Bronze (raw), Silver (cleaned), Gold (ML-ready) — for every source.
- Monthly snapshots from Jan 2023 to Dec 2024, idempotent and re-runnable.
- One Gold feature store + one Gold label store, joinable on (Customer_ID, snapshot_date).
- Zero target leakage: no post-application or repayment fields leak into features.
- Reproducible via docker-compose up and python main.py.

Raw data sources

Four CSVs, semicolon-delimited, monthly snapshots Jan 2023 – Dec 2024

File	Role	Grain	Key columns
feature_clickstream.csv	Feature	Customer × month	Customer_ID, snapshot_date, fe_1 ... fe_20
feature_attributes.csv	Feature	Customer × month	Customer_ID, Name, Age, SSN, Occupation, snapshot_date
feature_financials.csv	Feature	Customer × month	Customer_ID, Annual_Income, Credit_Mix, Outstanding_Debt, EMI, snapshot_date
lms_loan_daily.csv	Label only	Loan × day	loan_id, Customer_ID, installment_num, overdue_amt, due_amt, snapshot_date

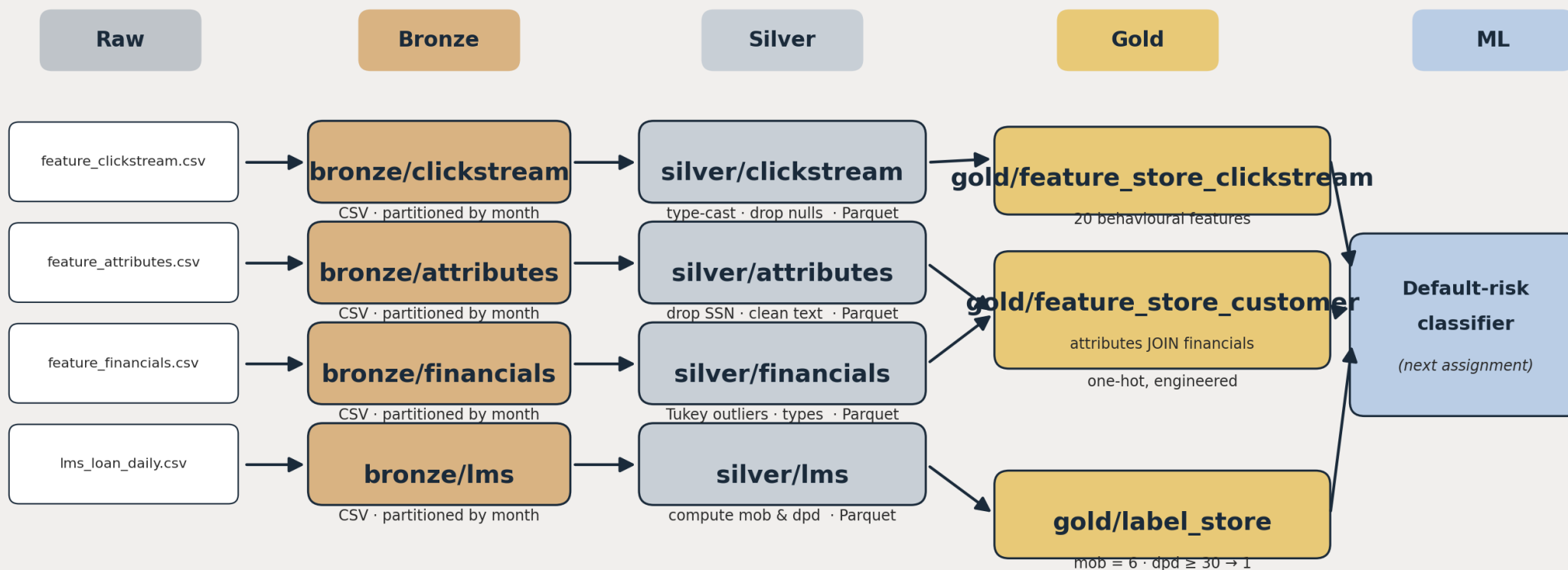
Why these splits matter for design

Three feature files have very different update cadences and quality profiles, so each gets its own Bronze / Silver pipeline. The loan file is repayment history — usable only for labels, never for features.

04

End-to-end architecture

Raw drop-zone → immutable Bronze → cleaned Silver → ML-ready Gold → downstream ML



Raw drop-zone → immutable Bronze partitions → cleaned Silver tables → ML-ready Gold feature & label stores

CSV

Raw + Bronze: human-readable, easy to inspect

Parquet

Silver + Gold: typed, compressed, columnar

Monthly partition

One file per snapshot_date — incremental runs

05

Bronze — raw landing zone

Slice once, store immutably, never transform

Three design choices

- Monthly partitions — one file per snapshot_date. Reprocessing is incremental, not full-history.
- CSV format preserved — keeps Bronze byte-identical to the source system so Silver fixes can never poison the raw record.
- Immutable on write — Bronze is the audit trail. Any Silver or Gold version can be reproduced from a frozen Bronze partition.

Filename pattern

```
datamart/bronze/<source>/  
  bronze_<source>_YYYY_MM_01.csv
```

Example

```
datamart/bronze/clickstream/  
  bronze_clickstream_2023_01_01.csv
```

Driven by

```
utils/data_processing_bronze_table.py  
  
main.py loops Jan-2023 → Dec-2024 and writes one partition per source per month.
```

No type casts, no filters, no joins at Bronze. The only operation is 'select * where snapshot_date = X'.

06

Silver — typed, cleaned, trustworthy

Parquet partitions with enforced schema and explicit data-quality rules

Issue found in raw	Treatment	Why
Mixed / wrong data types	Cast each column to its target type (Int, Float, Date, String)	Downstream feature engineering needs a stable schema
PII — SSN, Name	Drop SSN; keep Name only if needed for ops	Legal & privacy risk; SSN has no predictive value
Corrupted strings ("_____", "!@9#%8")	Drop the entire row	Looks like upstream encoding failure, not a missing value
Trailing underscores ("45_", "_")	Strip and re-cast	Recoverable noise — preserve the signal
Numeric outliers (Age = 8000, Income < 0)	Tukey IQR fences + domain bounds (Age 18–70)	Realistic loan applicants only; protects model from leverage points
Loan repayment fields (lms)	Compute mob (month-on-book) and dpd (days past due)	Engineered once at Silver so Gold label logic stays trivial

Output: datamart/silver/<source>/silver_<source>_YYYY_MM_01.parquet · one directory per month, schema-on-write.

Two tables, deliberately split by update cadence

feature_store_clickstream.parquet

20 numeric behaviour features (fe_1 ... fe_20)

Grain: Customer_ID × snapshot_date

Real-time-ish — refreshed independently from batch sources

No engineered columns; trust the raw signal

feature_store_customer.parquet

INNER JOIN: silver attributes ⋈ financials on (Customer_ID, snapshot_date)

One-hot: Occupation, Credit_Mix, Payment_Behaviour

Type_of_Loan exploded into n_loans_<type> counts

Slow-changing snapshot — batch refresh on schedule

Why two tables, not one?

- Different update cadence — joining them at Gold would force the slower table to wait, or staleness to leak in.
- Different ownership — attributes/financials come from core banking; clickstream comes from web analytics.
- Joined on demand at training time via (Customer_ID, snapshot_date) — keeps storage normalised, joins explicit.

Schema preview — feature_store_customer: Customer_ID · snapshot_date · Age · Annual_Income · Outstanding_Debt · EMI · Credit_Mix_Good · Credit_Mix_Standard · n_loans_Personal · n_loans_Auto · ...

One row per loan, binary default flag observed 6 months after origination

Label definition

Observation window

`mob = 6` (the 6th payment after loan start)

Default rule

`dpd >= 30 → label = 1` (defaulted)

`otherwise → label = 0` (current / paid)

Columns retained

`loan_id · Customer_ID · label · label_def · snapshot_date`

Why mob = 6 ?

- Short enough to label most loans in the 24-month window.
- Long enough that early-stage hiccups stop being noise.
- Standard industry vintage horizon for unsecured lending.

Code reuse

Label-store logic reused from Lab 2 verbatim — same Silver→Gold transformation on `lms_loan_daily`, same `mob/dpd` thresholds, same output schema.

Joinable to either `feature_store` table on (`Customer_ID`, `snapshot_date`) for ML training.

Data leakage controls

Rule: every feature must be knowable strictly before the loan application date

Target leakage

- Drop entire `lms_loan_daily` from features — it IS the answer (repayment history).
- Drop `Interest_Rate` — set by the underwriter after seeing the risk score; encodes the label.
- Re-check any 'Num_of_Delayed_Payment'-style fields: kept only when measured pre-application.

Temporal leakage

- `Snapshot_date` is the as-of cut. We never join a feature row from $t+1$ to a loan applied at t .
- Monthly partitions enforce the cut physically: a Jan-2023 training run only reads $\leq 2023-01-01$ files.
- Clickstream & financials are user-level snapshots — confirmed safe.

Train-test contamination

- Feature engineering is fit per partition, not across the whole table.
- No global means / scalars computed at Gold — that work belongs to the ML pipeline, after the split.
- Same `Customer_ID` can appear in train and test, but only at different `snapshot_dates`.

Sanity check (recommended by brief)

Fit a simple logistic regression on Gold `feature_store` $\bowtie$ `label_store`. If AUC is implausibly high (> 0.95) on held-out months, suspect a leaky column and trace it back through Silver.

Reproducible end-to-end in one command

Tech stack

Python 3

PySpark

Docker

JupyterLab

Parquet

Git / GitHub

How to run

1. `git clone <repo> && cd <repo>`

clone the project

2. `docker-compose build`

build the Spark + Jupyter image

3. `docker-compose up`

launches JupyterLab on <http://localhost:8888>

4. `python main.py`

loops Jan-23 → Dec-24, writes `datamart/{bronze,silver,gold}`

5. Inspect `datamart/gold/` for ML-ready Parquet tables

feature & label stores land here

Repository

github.com/<your-handle>/MLE_ASM1

(see `Readme.txt` in submission zip)

Deliverables in zip

- `main.py`
- `Dockerfile` · `docker-compose.yaml`
- `requirements.txt`
- `utils/` · `data/`
- `Readme.txt` (repo link)